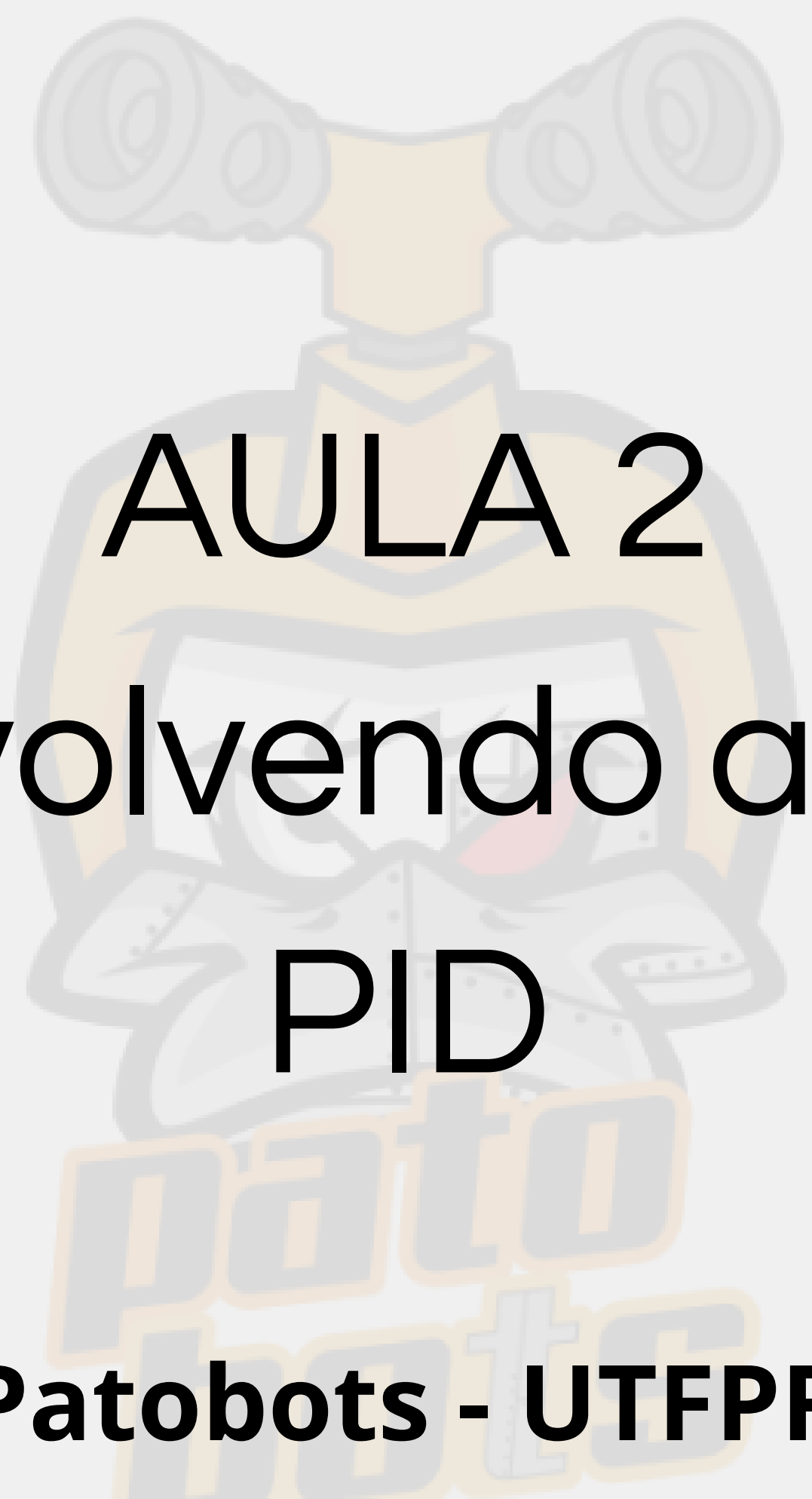




AULA 2

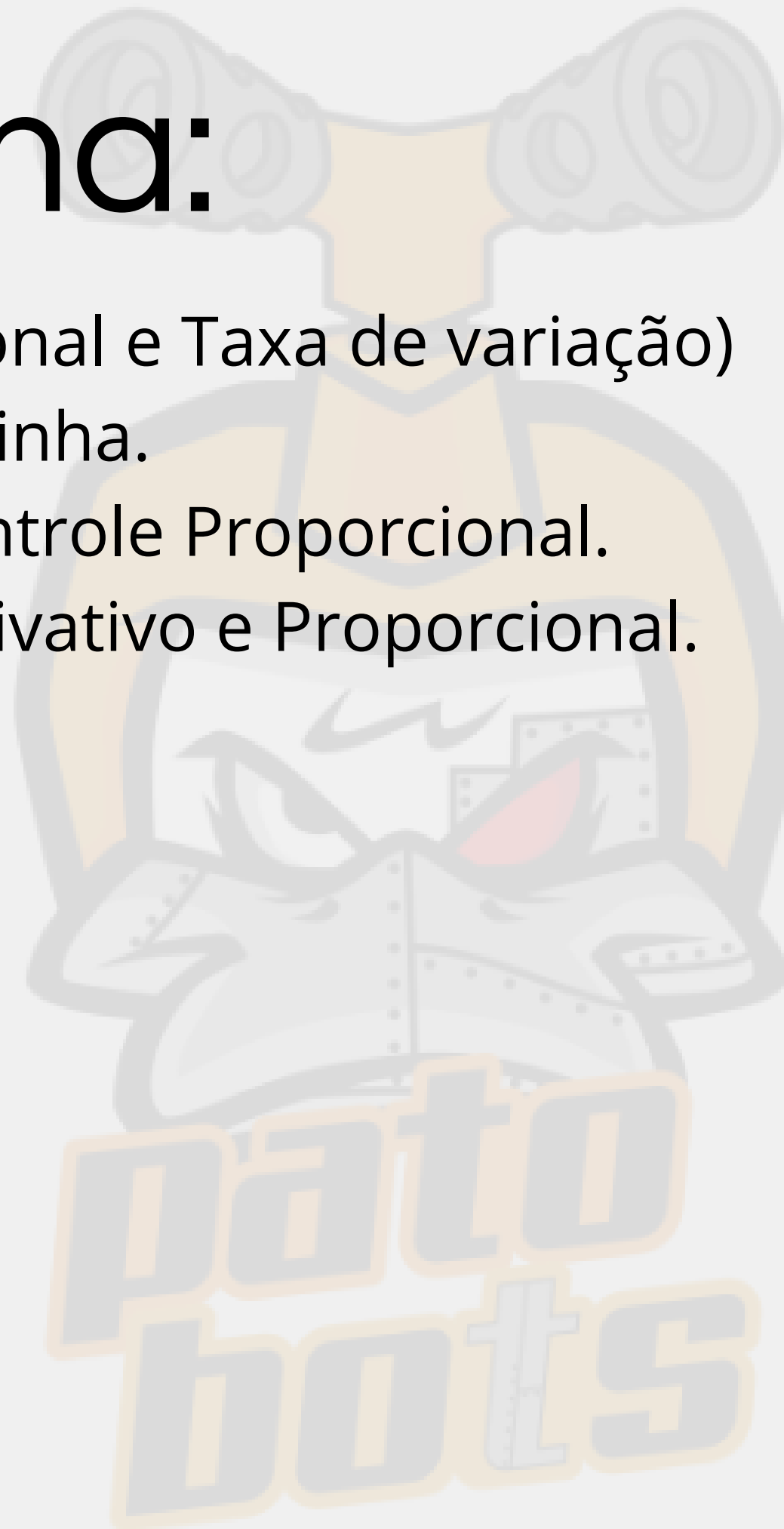
Desenvolvendo a Lógica

PID

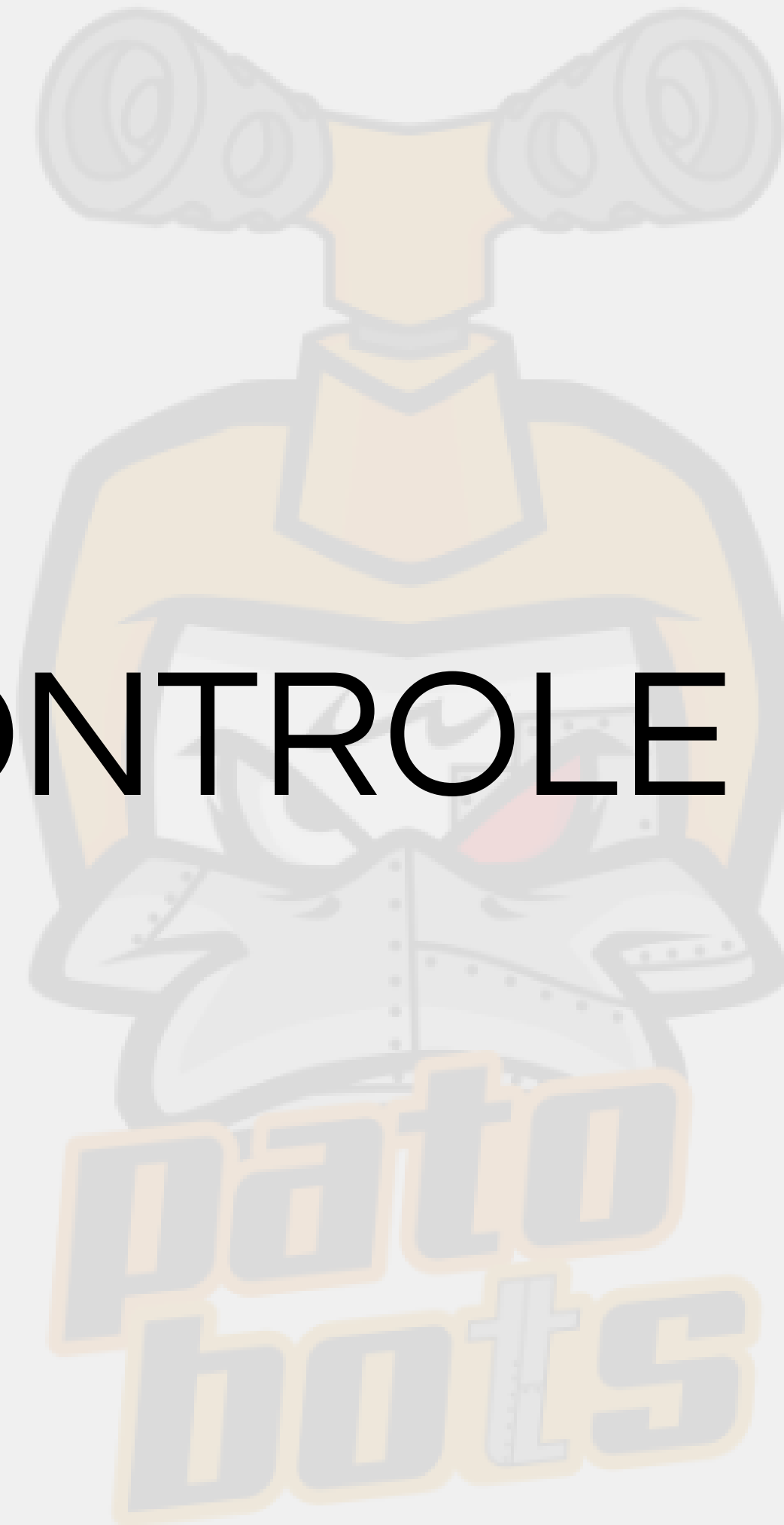
Patobots - UTFPR

Cronograma:

- Controle PID. (Proporcional e Taxa de variação)
- Lógica do Seguidor de Linha.
- Teste com apenas o Controle Proporcional.
- Teste com Controle Derivativo e Proporcional.

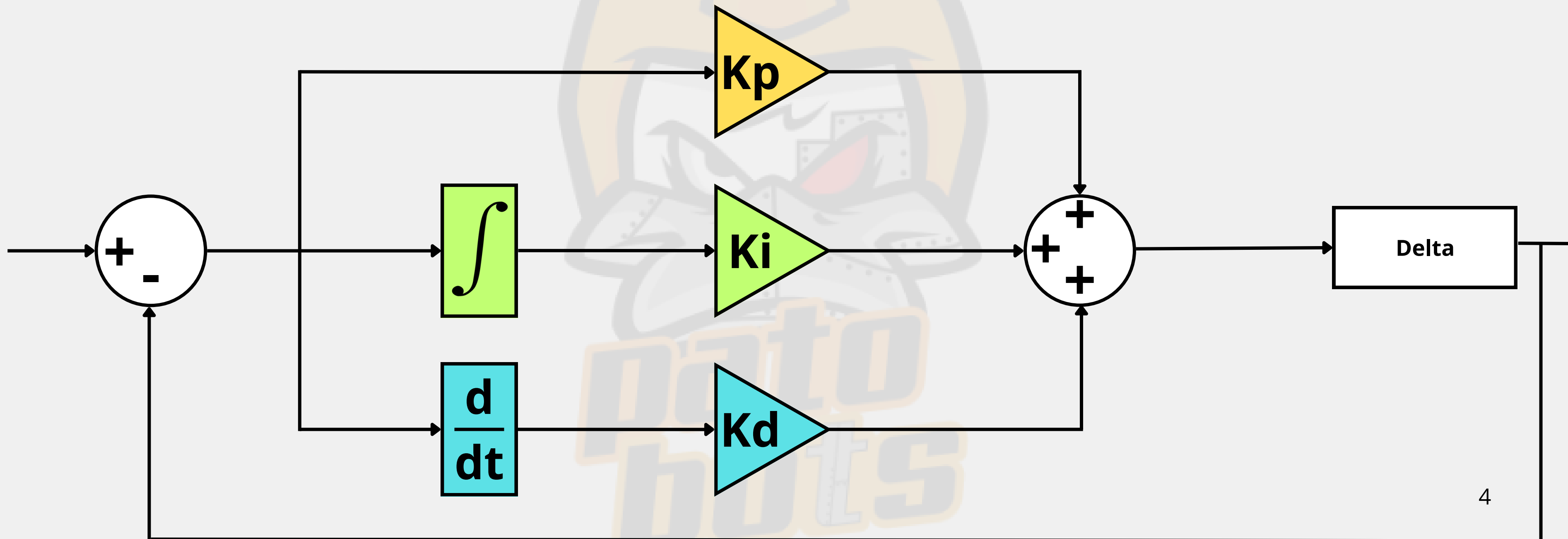


CONTROLE PID



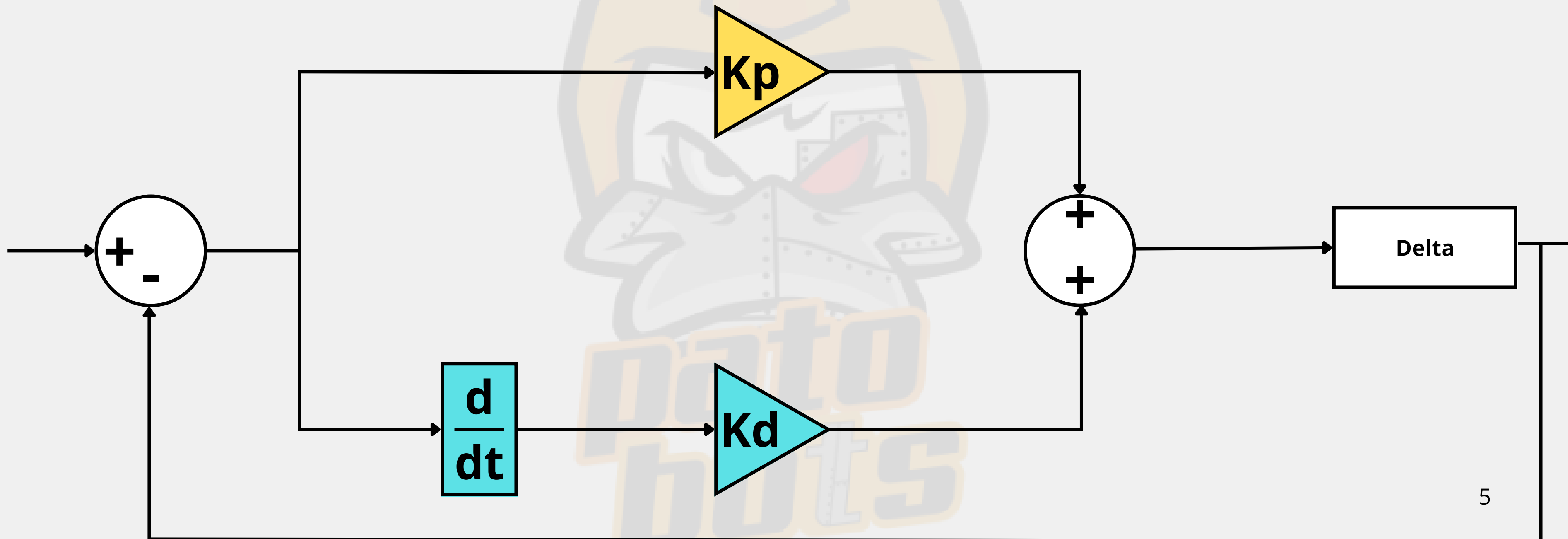
Controle PID

(Proporcional, Integral e Derivativo)



Controle PID

(Proporcional, Integral e Derivativo)



Controle PID

Controle PID para Robôs:

- **Estabilização:** O Controle ajuda o robô a manter equilíbrio e seguir o caminho correto.
- **Seguidor de Linha:** Ideal para robôs que precisam seguir trajetórias marcadas no chão.



Controle PID

Fórmula de Controle P e D:

Ajusta a movimentação do robô com base no erro de posição:

- **P (Proporcional):** Reage ao erro atual.
- **D (Derivativo):** Considera a variação do erro para corrigir a velocidade.

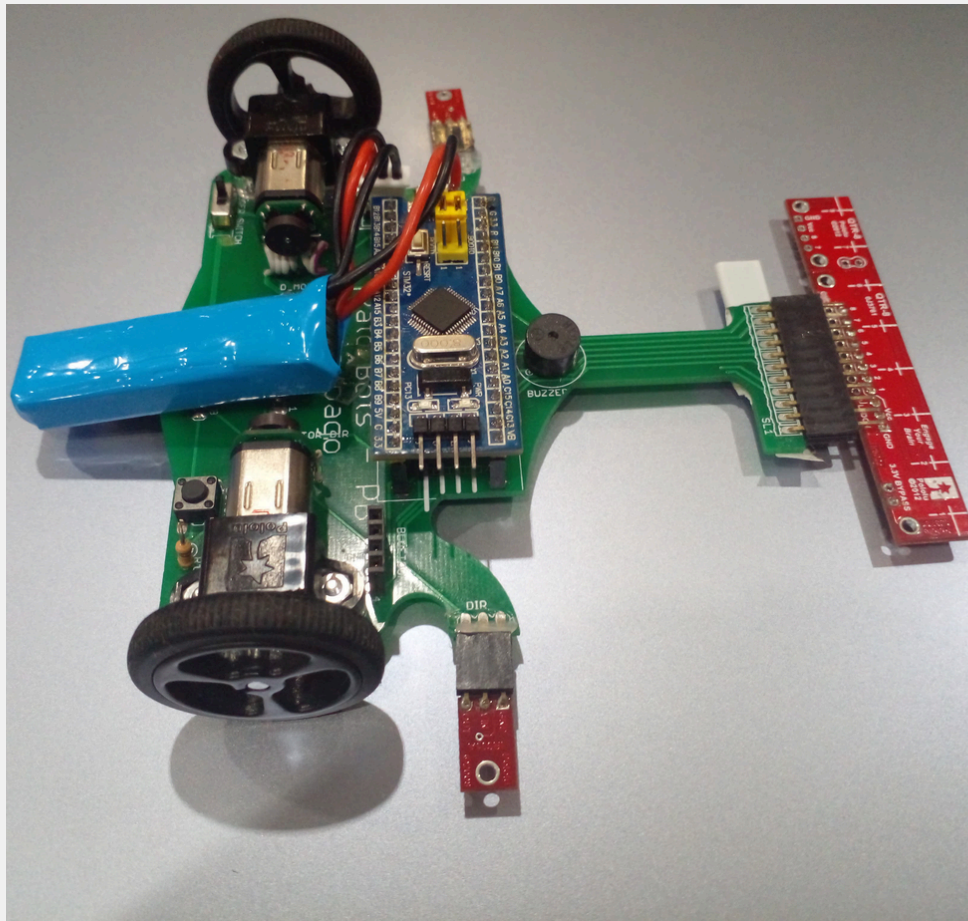
- **Fórmula:**

$$\text{Delta} = K_p * \text{Erro Atual} + K_d * (\text{Erro Atual} - \text{Erro Anterior})$$

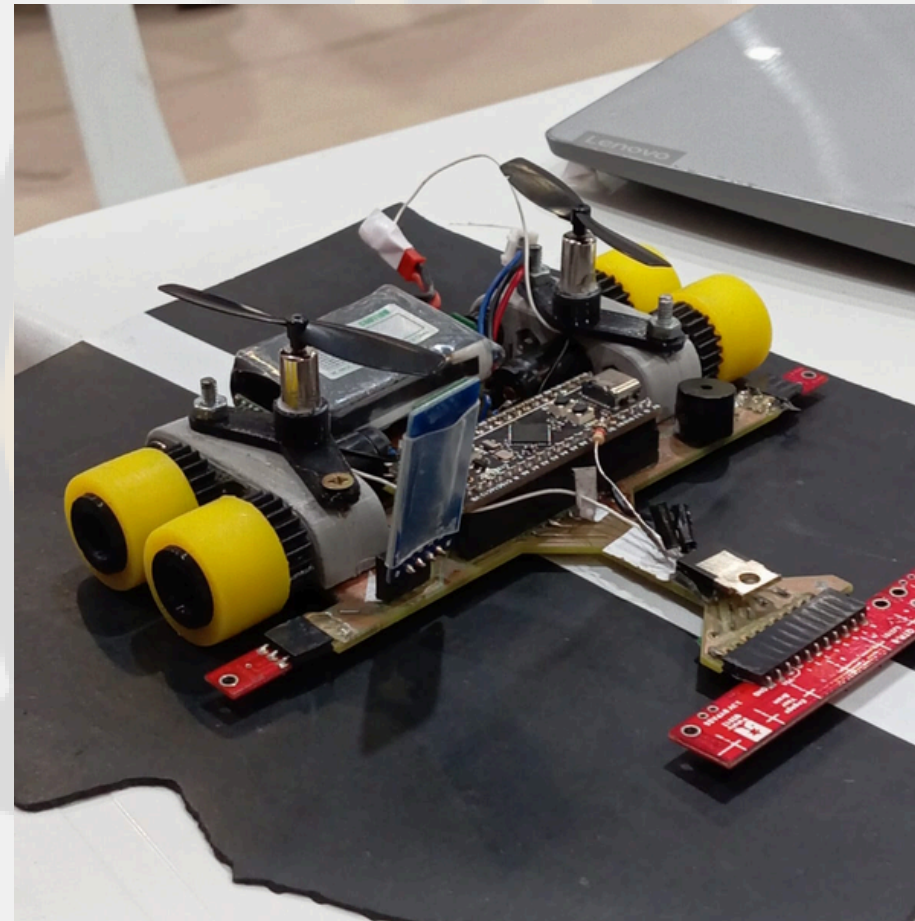
SEGUIDOR DE LINHA



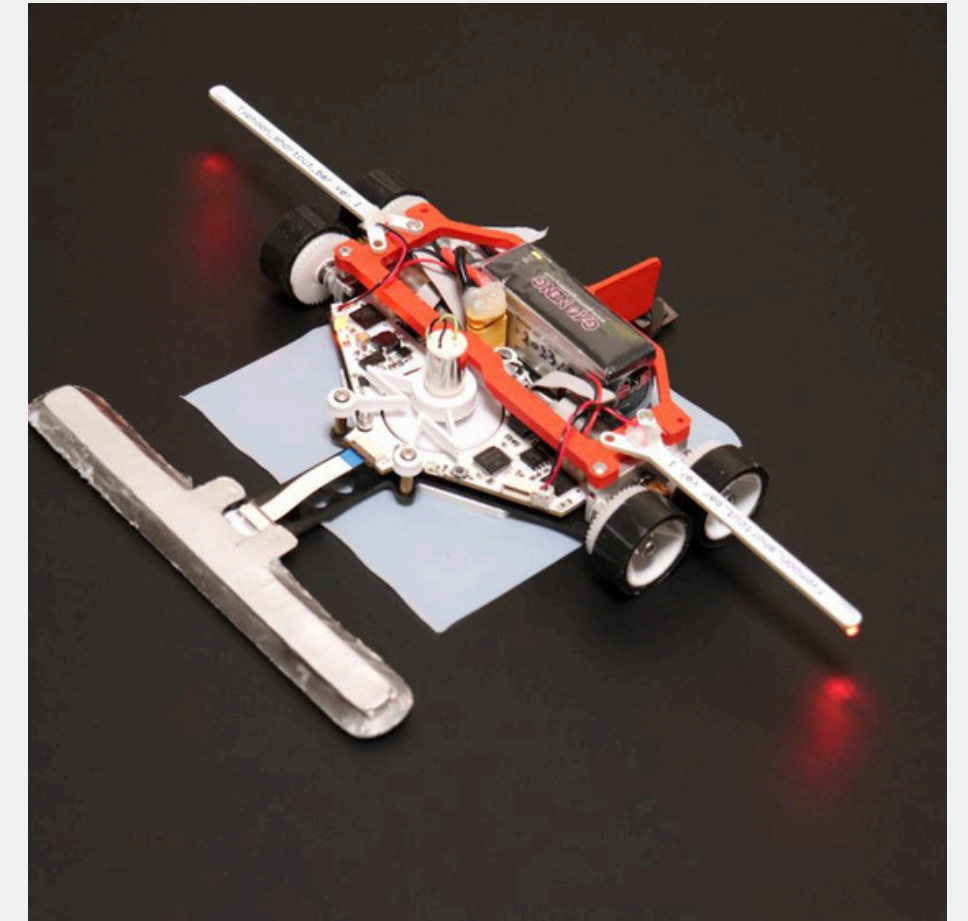
Tipos de Seguidor de Linha PRO



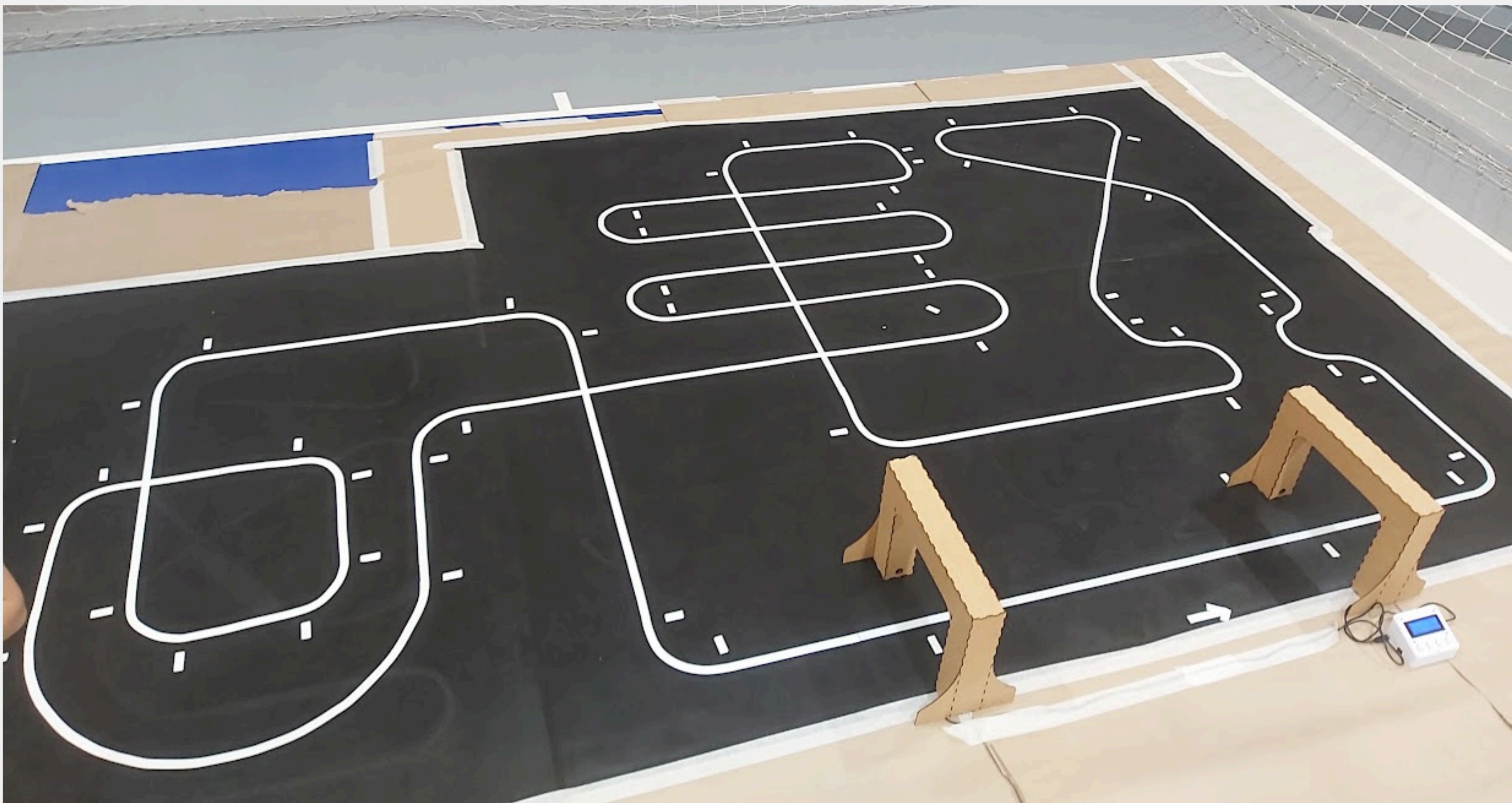
Básico



Com Ventoinha



Com Sucção



Lógica do Seguidor de Linha

- O robô detecta uma faixa no chão e ajusta sua trajetória para permanecer sobre ela.
- Seus sensores identificam a posição da linha e enviam sinais para o **Sistema de Controle**.
- O **Sistema de Controle** ajusta os motores com base nos dados dos sensores, corrigindo sua direção.





CÓDIGO DE CONTROLE DO SEGUIDOR (Proporcional)

```
1 // Configuração dos motores
2 const int motor1PWM = 9; // Pino PWM para o motor 1
3 const int motor1DirA = 8; // Pino de controle de direção A do motor 1
4 const int motor1DirB = 7; // Pino de controle de direção B do motor 1
5
6 const int motor2PWM = 10; // Pino PWM para o motor 2
7 const int motor2DirA = 12; // Pino de controle de direção A do motor 2
8 const int motor2DirB = 11; // Pino de controle de direção B do motor 2
9
10 // Definindo os pinos das entradas analógicas
11
12 const int pinA0 = A0;
13 const int pinA1 = A1;
14 const int pinA2 = A2;
15
16 long int faixa = 0;
17 long int ref = 1000;
18 long int erro = 0;
19 long int valorA0, valorA1, valorA2;
20 long int num, den;
21
22 long int delta;
23
24 int velMotor_dir = 100;
25 int velMotor_esq = 100;
26 float Kp = 0.1;
27
```

```
27
28 void setup() {
29
30     Serial.begin(9600);
31
32     pinMode(motor1PWM, OUTPUT);
33     pinMode(motor1DirA, OUTPUT);
34     pinMode(motor1DirB, OUTPUT);
35
36     pinMode(motor2PWM, OUTPUT);
37     pinMode(motor2DirA, OUTPUT);
38     pinMode(motor2DirB, OUTPUT);
39
40 }
41
```

```
41
42 void loop() {
43
44     // Lê os valores das entradas analógicas
45     valorA0 = analogRead(pinA0);
46     valorA1 = analogRead(pinA1);
47     valorA2 = analogRead(pinA2);
48
49     // Calculo para ver a direção que o carrinho deve ajustar
50     num = 0*valorA0 + 1000*valorA1 + 2000*valorA2;
51     den = valorA0 + valorA1 + valorA2;
52
53     faixa = num/den;
54
55     erro = ref - faixa;
56
```

```
56
57  if(erro > 0){
58      delta = Kp*(float)erro;
59      // A Velocidade não pode ser maior que 255
60      if(delta > 255) delta = 255;
61
62      // Movimenta o motor da direita re
63      digitalWrite(motor1DirA, LOW);
64      digitalWrite(motor1DirB, HIGH);
65      analogWrite(motor1PWM, delta);
66
67      // Movimenta o motor da esquerda para a frente
68      digitalWrite(motor2DirA, HIGH);
69      digitalWrite(motor2DirB, LOW);
70      analogWrite(motor2PWM, delta);
71
72  }
```

```
72     }
73     else{
74         delta = -Kp*(float)erro;
75         if (delta > 255) delta = 255;
76
77         // Movimenta o motor da direita frente
78         digitalWrite(motor1DirA, HIGH);
79         digitalWrite(motor1DirB, LOW);
80         analogWrite(motor1PWM, delta);
81
82         // Movimenta o motor da esquerda re
83         digitalWrite(motor2DirA, LOW);
84         digitalWrite(motor2DirB, HIGH);
85         analogWrite(motor2PWM, delta);
86
87     }
88 }
```



CÓDIGO DE CONTROLE DO SEGUIDOR (Derivativo)

```
55   faixa = num/den;
56
57   erro = ref - faixa;
58
59   if(erro > 0){
60       delta = Kp*(float)erro + Kd*(float)(erro - erroAnterior);
61       erroAnterior = erro;
62       // A Velocidade não pode ser maior que 255
63       if(delta > 255) delta = 255;
64
65       // Movimenta o motor da direita re
66       digitalWrite(motor1DirA, LOW);
67       digitalWrite(motor1DirB, HIGH);
68       analogWrite(motor1PWM, delta);
69
70       // Movimenta o motor da esquerda para a frente
71       digitalWrite(motor2DirA, HIGH);
72       digitalWrite(motor2DirB, LOW);
73       analogWrite(motor2PWM, delta);
74
75   }
```

```
75     }
76     else{
77         delta = -(Kp*(float)erro + Kd*(float)(erro - erroAnterior));
78         erroAnterior = erro;
79         if (delta > 255) delta = 255;
80
81         // Movimenta o motor da direita frente
82         digitalWrite(motor1DirA, HIGH);
83         digitalWrite(motor1DirB, LOW);
84         analogWrite(motor1PWM, delta);
85
86         // Movimenta o motor da esquerda re
87         digitalWrite(motor2DirA, LOW);
88         digitalWrite(motor2DirB, HIGH);
89         analogWrite(motor2PWM, delta);
90
91     }
92 }
```